OpenMesh User Guide

Version 0.1.0 · **April 2026** · open-mesh.ai

---

1. What Is OpenMesh?

OpenMesh is an AI-native operations platform. You declare **what** your infrastructure should do using YAML goals, and OpenMesh figures out **how** — using AI reasoning, event-driven automation, and multi-channel integrations.

Core concepts:

Concept	Role
Observers	Watch your world (cron, HTTP health, GitHub webhooks, Slack, log files) and emit events
EventBus	Durable WAL-backed event pipeline with glob routing and deduplication
GoalEngine	Loads YAML goals, matches events to patterns, executes step sequences
Operators	Execute actions — code analysis, notifications, infrastructure commands, data queries, AI reasoning
AI Engine	LLM-powered goal interpretation, anomaly detection, planning, and interactive refinement
MCP Server	Exposes operators as MCP tools for Claude, Cursor, VS Code, or any MCP client
Channels	Bridge messages across Slack, Discord, Telegram, and webhooks
Telemetry	OpenTelemetry traces + metrics, Pino structured logging, Prometheus /metrics
Plugins	Dynamically load observers/operators/goals from npm or local directories
Dashboard	Web UI with live SSE — goals, events, execution log

Architecture:

```
Observers ──→ EventBus (WAL) ──→ GoalEngine ──→ Operators
  cron           durable JSONL       YAML goals     code
  http-health    glob matching       conditions     comms
  github         dedup               templates      infra
  slack          anomaly detect      escalation     data
  log-stream     ↑                   AI planning    AI
  channels ←──────┘                                 channels
```

---

2. Installation

Prerequisites

- **Node.js** ≥ 22
- **pnpm** ≥ 10

From Source

```
git clone https://github.com/thebotclub/openmesh.git
cd openmesh
pnpm install
pnpm build
```

With Docker

```
git clone https://github.com/thebotclub/openmesh.git
cd openmesh
docker compose up
```

This starts three services: **mesh** (main runtime), **LiteLLM** (LLM proxy with 100+ providers), and **OpenTelemetry collector**.

---

3. Quick Start

```
# 1. Initialize a project (creates goals/ directory + example goal)
mesh init

# 2. Run the mesh
mesh run

# 3. Inject a test event
mesh inject cron.tick

# 4. Check status
mesh status

# 5. View event log
mesh logs
```

For the full stack with AI, channels, telemetry, and dashboard:

```
mesh run --ai --channels --telemetry --dashboard --dashboard-port 3777
```

---

4. CLI Reference

`mesh init`

Initialize an OpenMesh project in the current directory. Creates:

- `goals/` — directory for YAML goal definitions
- `.openmesh/` — data/persistence directory
- `mesh.config.json` — configuration file
- Example goal YAML file

`mesh run [options]`

Start the mesh runtime.

Flag	Default	Description
`-g, --goals <dir>`	`goals`	Goals directory path
`-d, --data <dir>`	`.openmesh`	Data/persistence directory
`--log-level <level>`	`info`	Log level: `debug`, `info`, `warn`, `error`
`--dashboard`	`false`	Start web dashboard

`--dashboard-port <port>`	`3777`	Dashboard HTTP port
`--ai`	`false`	Enable AI operator & anomaly detection
`--telemetry`	`false`	Enable OpenTelemetry traces/metrics
`--channels`	`false`	Enable multi-channel messaging
`--plugins <dirs...>`	—	Load plugins from local directories

`mesh inject <type> [options]`

Inject a test event into the mesh.

Flag	Default	Description
`-p, --payload <json>`	`{}`	Event payload as JSON string
`-s, --source <source>`	`manual`	Event source identifier
`-g, --goals <dir>`	`goals`	Goals directory
`-d, --data <dir>`	`.openmesh`	Data directory

Example:

```
mesh inject github.pr.opened \
  -p '{"repo":"my-app","action":"opened"}' \
  -s webhook
```

`mesh status [options]`

Show mesh status — loaded goals, recent events, pending approvals.

Flag	Default	Description
`-d, --data <dir>`	`.openmesh`	Data directory
`-g, --goals <dir>`	`goals`	Goals directory

`mesh logs [options]`

Stream the event log from `events.wal.jsonl`.

Flag	Default	Description
`-d, --data <dir>`	`.openmesh`	Data directory
`-n, --lines <count>`	`20`	Number of recent lines
`-t, --type <pattern>`	—	Filter by event type (substring match)

`mesh ai interpret <text> [options]`

Convert natural language to goal YAML via LLM.

Flag	Default	Description
`--model <model>`	config default	LLM model override

`--save`	`false`	Save generated goal to goals directory
`-g, --goals <dir>`	`goals`	Goals directory

Examples:

```
mesh ai interpret "When GitHub push fails CI, analyze logs and notify #engineering"
mesh ai interpret "Monitor HTTP health every 5 min and restart if down" --save
```

`mesh ai analyze [options]`

Detect anomalies in recent events using AI.

Flag	Default	Description
`-d, --data <dir>`	`.openmesh`	Data directory
`-w, --window <count>`	`50`	Number of recent events to analyze

Output: Anomalies with severity, type, description, and suggested actions.

`mesh ai refine [options]`

Interactive multi-turn goal refinement session (REPL).

Flag	Default	Description
`--model <model>`	config default	LLM model override
`--save`	`false`	Save final goal
`-g, --goals <dir>`	`goals`	Goals directory

Flow: Describe goal → AI interprets → provide feedback → iterate → save.

`mesh mcp serve [options]`

Expose mesh operators as MCP tools (stdio transport for Claude Desktop, Cursor, VS Code).

Flag	Default	Description
`-d, --data <dir>`	`.openmesh`	Data directory
`-g, --goals <dir>`	`goals`	Goals directory

Exposed tools: `mesh_op_<operatorId>`, `mesh_inject`, `mesh_status`, `mesh_goals`

`mesh mcp connect <command...> [options]`

Connect an external MCP server and import its tools as mesh operators.

```
mesh mcp connect npx -y @modelcontextprotocol/server-filesystem /tmp
```

`mesh channels list`

List configured channels from `mesh.config.json`.

`mesh channels test <channel> <message>`

Send a test message to a specific channel adapter.

```
mesh channels test slack "Hello from OpenMesh!"
```

`mesh plugin load <path>`

Load and inspect a local plugin directory (validates manifest, shows metadata).

`mesh plugin install <name>`

Install a plugin from npm and register it.

---

5. Writing Goals (YAML)

Goals live in `goals/*.yaml`. They declare **what events to watch** and **what steps to take**.

Complete Goal Schema

```yaml
id: build-repair                            # REQUIRED — unique ID
description: Investigate CI build failures  # REQUIRED — human description

observe:                                    # REQUIRED — event patterns
  - type: "ci.build.failed"                #   glob pattern (*, **)
    where:                                  #   optional payload filter
      repo: my-app

then:                                       # REQUIRED — execution steps
  - label: investigate                      #   REQUIRED — step ID
    operator: code                          #   REQUIRED — operator name
    task: "Investigate: {{event.payload.error}}"  # REQUIRED — task (templated)
    when: "previous.status == 'success'"    #   optional condition
    timeoutMs: 30000                        #   optional timeout (ms)
    channel: slack                          #   optional channel target
    to: "#oncall"                           #   optional recipient
    retry:                                  #   optional retry policy
      maxRetries: 3
      delayMs: 1000
      backoffMultiplier: 2.0
      maxDelayMs: 60000

  - label: notify
    operator: comms
    task: "Report: {{steps.investigate.summary}}"
    when: "investigate.status == 'success'"

escalate:                                   # OPTIONAL — auto-escalation
  afterFailures: 3
  channel: slack
  to: "#critical"

dedupWindowMs: 5000                         # OPTIONAL — dedup window (ms)
```

Template Variables

Variable	Description
`{{event.type}}`	Event type string
`{{event.source}}`	Event source identifier
`{{event.timestamp}}`	ISO-8601 timestamp
`{{event.payload.<key>}}`	Any event payload field
`{{steps.<label>.status}}`	Prior step status: `success`, `failure`, `timeout`, `denied`
`{{steps.<label>.summary}}`	Prior step summary text
`{{steps.<label>.data.<key>}}`	Prior step structured data

Available Operators

Operator	Description	Example Task
`code`	Code analysis, search, test runner, git diff	`"Analyze test failures in {{event.payload.repo}}"`
`comms`	Send notifications (stdout, channels)	`"Build failed for {{event.source}}"`
`infra`	Execute infrastructure commands (allowlisted)	`"exec: kubectl rollout restart deployment/app"`
`data`	File read, line count, grep, directory stats	`"read: /var/log/app.log"`
`ai`	LLM reasoning and analysis	`"Analyze this error: {{event.payload.error}}"`
`channels`	Send to Slack/Discord/Telegram/webhook	`"Send to slack: Build failed"`

Example Goals

Service health monitoring:

```yaml
id: service-health
description: Monitor HTTP endpoints and alert on failures
observe:
  - type: http-health.down
then:
  - label: diagnose
    operator: ai
    task: "Diagnose why {{event.payload.url}} is down"
  - label: alert
    operator: channels
    task: "Send to slack: {{event.payload.url}} is DOWN — {{steps.diagnose.summary}}"
    when: "diagnose.status == 'success'"
escalate:
  afterFailures: 5
  channel: slack
  to: "#oncall"
```

Code quality gate:

```yaml
id: code-quality
description: Review PRs for quality issues
observe:
```

```
  - type: github.pr.opened
then:
  - label: review
    operator: code
    task: "Analyze PR #{{event.payload.number}} in {{event.payload.repo}}"
  - label: comment
    operator: comms
    task: "PR Review: {{steps.review.summary}}"
```

Incident response:

```
id: incident-response
description: Auto-triage production incidents
observe:
  - type: log-stream.error
    where:
      severity: critical
then:
  - label: investigate
    operator: code
    task: "Investigate critical error: {{event.payload.message}}"
  - label: gather-data
    operator: data
    task: "read: {{event.payload.logFile}}"
    when: "investigate.status == 'success'"
  - label: notify
    operator: channels
    task: "Send to slack: CRITICAL — {{steps.investigate.summary}}"
escalate:
  afterFailures: 1
  channel: slack
  to: "#incidents"
```

---

6. AI Engine Configuration

OpenMesh uses the OpenAI SDK pointed at any OpenAI-compatible endpoint. This works out of the box with **LiteLLM** (100+ providers), **Ollama**, **vLLM**, **OpenAI**, **Anthropic**, **OpenRouter**, and more.

Environment Variables

Variable	Default	Description
`OPENMESH_LLM_BASE_URL`	`http://localhost:4000/v1`	LLM API endpoint
`OPENMESH_LLM_API_KEY`	`not-needed`	API key
`OPENMESH_LLM_MODEL`	`gpt-4o-mini`	Default model
`OPENAI_API_KEY`	—	Fallback for API key

Programmatic Config

```
import { AIEngine } from '@openmesh/ai';

const engine = new AIEngine({
```

```
  baseUrl: 'http://localhost:4000/v1',
  apiKey: 'sk-...',
  model: 'gpt-4o',
  temperature: 0.2,    // 0-1
  maxTokens: 4096,
});

// Single-turn prompt
const answer = await engine.prompt(
  'You are a DevOps expert.',
  'Why is my Kubernetes pod crashlooping?'
);

// Structured JSON response
const result = await engine.promptJSON<{ severity: string }>(
  'Classify the severity.',
  'CPU at 98% for 30 minutes'
);
```

RAG Context

Feed event/checkpoint history into AI prompts for better reasoning:

```
import { RAGContextBuilder } from '@openmesh/ai/ragContext';

const context = new RAGContextBuilder()
  .addEvents(recentEvents)
  .addCheckpoints(stateCheckpoints)
  .addGoalStates(activeGoals)
  .build();

// Pass to interpreter or planner
await interpreter.interpret(description, { ragContext: context });
await planner.plan(goalId, { ragContext: context });
```

Interactive Refinement

```
import { RefineSession } from '@openmesh/ai/refineSession';

const session = new RefineSession(engine);
const initial = await session.start('Monitor my APIs and alert on failures');
console.log(initial.yaml);

const refined = await session.refine('Add retry with exponential backoff');
console.log(refined.yaml);

const yaml = session.toYaml();  // Final goal YAML string
```

---

7. MCP Integration

Expose Mesh as MCP Server

```
# stdio transport (Claude Desktop, Cursor, VS Code)
mesh mcp serve

# Programmatic with HTTP transport (SSE + POST)
```

```
import { MeshMCPServer } from '@openmesh/mcp';

const mcpServer = new MeshMCPServer(mesh, {
  name: 'openmesh',
  version: '0.1.0',
  exposeOperators: ['code', 'comms', 'data'],  // allowlist (default: all)
  enableInject: true,
  enableGoals: true,
});

// stdio (for MCP clients)
await mcpServer.start();

// HTTP transport (SSE + POST, for remote clients)
await mcpServer.startHttp({
  port: 4000,
  hostname: '0.0.0.0',
  cors: true,
  apiKey: 'my-secret-key',
});
```

HTTP transport routes:

- `POST /messages` — Send JSON-RPC messages
- `GET /sse` — Server-Sent Events stream
- `GET /health` — Health check (no auth required)

Import External MCP Tools

```
mesh mcp connect npx -y @modelcontextprotocol/server-filesystem /tmp
```

```
import { MeshMCPClient } from '@openmesh/mcp';

const client = new MeshMCPClient(mesh);
await client.connect('npx', ['-y', '@modelcontextprotocol/server-filesystem', '/tmp']);
// External tools are now available as mesh operators
```

Claude Desktop Configuration

Add to `claude_desktop_config.json` :

```
{
  "mcpServers": {
    "openmesh": {
      "command": "mesh",
      "args": ["mcp", "serve"]
    }
```

```
  }
}
```

---

8. Multi-Channel Messaging

Configuration (`mesh.config.json`)

```
{
  "channels": {
    "slack": {
      "botToken": "xoxb-...",
      "defaultChannel": "#ops",
      "pollIntervalMs": 5000
    },
    "discord": {
      "botToken": "...",
      "guildId": "...",
      "defaultChannelId": "..."
    },
    "telegram": {
      "botToken": "...",
      "defaultChatId": "...",
      "pollIntervalMs": 3000
    },
    "webhook": {
      "port": 3780,
      "outboundUrl": "https://hooks.example.com/mesh"
    }
  }
}
```

Environment Variable Fallbacks

Adapter	Variables
Slack	`SLACK_BOT_TOKEN`, `SLACK_DEFAULT_CHANNEL`
Discord	`DISCORD_BOT_TOKEN`, `DISCORD_DEFAULT_CHANNEL`, `DISCORD_GUILD_ID`
Telegram	`TELEGRAM_BOT_TOKEN`, `TELEGRAM_DEFAULT_CHAT_ID`
Webhook	`OPENMESH_WEBHOOK_PORT`, `OPENMESH_WEBHOOK_OUTBOUND_URL`

Use in Goals

```
then:
  - label: alert
    operator: channels
    task: "Send to slack: Build {{event.payload.repo}} failed"
  - label: page
    operator: channels
    task: "Send to telegram: CRITICAL — {{event.payload.error}}"
```

Programmatic Usage

```
import { ChannelRouter } from '@openmesh/channels';

const router = new ChannelRouter();
router.register(new SlackChannel(config));
router.register(new DiscordChannel(config));

await router.start();
await router.send({ channel: 'slack', text: 'Hello!', sender: 'mesh', id: '...', timestamp: new
Date().toISOString() });
```

---

9. Telemetry & Observability

Configuration

```
import { MeshTelemetry } from '@openmesh/telemetry';

const telemetry = new MeshTelemetry({
  serviceName: 'openmesh',
  otlpEndpoint: 'http://localhost:4318',
  logLevel: 'info',
  prettyLogs: true,          // dev mode
  enableTracing: true,
  enableMetrics: true,
  prometheusPort: 9090,      // optional Prometheus endpoint
});
```

Environment Variables

Variable	Description
`OTEL_EXPORTER_OTLP_ENDPOINT`	OTLP collector URL (overrides config)

Prometheus Metrics

When `prometheusPort` is set, an HTTP server exposes metrics at `GET /metrics` in Prometheus exposition format:

```
# HELP openmesh_events_total Total events processed
# TYPE openmesh_events_total counter
openmesh_events_total 142

# HELP openmesh_goal_executions_total Goal executions by status
# TYPE openmesh_goal_executions_total counter
openmesh_goal_executions_total{status="success"} 98
openmesh_goal_executions_total{status="failure"} 12
```

With Docker

The `docker-compose.yml` includes an OpenTelemetry collector that receives traces and metrics from the mesh runtime.

```
docker compose up
```

```
# Traces/metrics available at the collector: http://localhost:4318
```

---

10. Dashboard

Launch

```
mesh run --dashboard --dashboard-port 3777
# Open http://localhost:3777
```

Authentication

If API keys are configured, all dashboard routes require authentication:

```
import { startDashboard } from '@openmesh/dashboard';

startDashboard({
  port: 3777,
  apiKeys: ['my-secret-key'],  // optional
  dataDir: '.openmesh',
  goalsDir: 'goals',
});
```

Authenticate via:

- `Authorization: Bearer <key>` header
- `X-Mesh-Api-Key: <key>` header
- `?apiKey=<key>` query parameter
- `mesh_api_key` cookie

API Endpoints

Route	Description
`GET /`	HTML dashboard UI
`GET /api/state`	JSON: goals, events, checkpoints
`GET /api/stream`	SSE: live event stream

Dashboard Panels

- **Goals** — loaded YAML goals with observation patterns and steps
- **Recent Events** — live event stream from the WAL
- **Execution Log** — state checkpoints (matches, step results, completions)

---

11. Plugins

Plugin Manifest (`openmesh-plugin.json`)

```
{
  "name": "openmesh-plugin-datadog",
  "version": "1.0.0",
  "type": "observer",
  "entry": "./dist/index.js",
```

```
  "description": "Datadog metrics observer",
  "permissions": ["api:datadog", "network:https"],
  "config": {
    "apiKey": "...",
    "site": "us1"
  }
}
```

Field	Required	Description
`name`	Yes	Plugin name
`version`	Yes	Semantic version
`type`	Yes	`observer`, `operator`, `goal`, or `bundle`
`entry`	Yes	Path to main JS/TS file
`description`	No	Human-readable description
`permissions`	No	Requested permissions
`config`	No	Default configuration

Plugin Entry File

```
// Single observer plugin
import { defineObserver } from '@openmesh/sdk';

export default defineObserver({
  id: 'datadog',
  init: (bus) => {
    // Poll Datadog API and emit events
    setInterval(async () => {
      const metrics = await fetchDatadogMetrics();
      bus.emit({ type: 'datadog.metric', source: 'datadog', payload: metrics });
    }, 60000);
  }
});
```

```
// Bundle plugin (multiple components)
export default {
  observers: [myObserver],
  operators: [myOperator],
  goals: [myGoal],
};
```

Loading Plugins

```
# Local directory
mesh plugin load ./my-plugin

# From npm
mesh plugin install openmesh-plugin-kubernetes
```

```
# Run with plugins
mesh run --plugins ./my-plugin ./another-plugin
```

---

12. Docker Deployment

Services

Service	Port	Description
`mesh`	3000 (dashboard), 4000 (MCP)	OpenMesh runtime
`litellm`	4001	LLM proxy (100+ model providers)
`otel-collector`	4318	OpenTelemetry collector

Environment Variables

```
# Required for AI features
OPENAI_API_KEY=sk-...

# Optional channel webhooks
SLACK_WEBHOOK_URL=https://hooks.slack.com/...
DISCORD_WEBHOOK_URL=https://discord.com/api/webhooks/...
TELEGRAM_BOT_TOKEN=...
TELEGRAM_CHAT_ID=...
```

Running

```
# Start all services
docker compose up -d

# View logs
docker compose logs -f mesh

# Stop
docker compose down
```

---

13. SDK Reference

`defineObserver(config)`

```
import { defineObserver } from '@openmesh/sdk';

export default defineObserver({
  id: 'my-observer',
  init: (bus) => {
    // bus.emit({ type, source, payload })
  }
});
```

`defineOperator(config)`

```
import { defineOperator } from '@openmesh/sdk';

export default defineOperator({
  id: 'my-operator',
  exec: async (task, context) => {
    // task: string (the task description from YAML)
    // context: { event, mesh, approver? }
    return { status: 'success', summary: '...', data: {} };
  }
});
```

`defineGoal(config)`

```
import { defineGoal } from '@openmesh/sdk';

export default defineGoal({
  id: 'my-goal',
  description: '...',
  observe: [{ type: 'cron.tick' }],
  then: [{ label: 'step1', operator: 'comms', task: 'Hello' }],
});
```

---

14. Persistent State

All data is stored as JSONL files in the `.openmesh/` directory:

File	Contents
`events.wal.jsonl`	Durable write-ahead log of all events
`state.jsonl`	Execution state (goal matches, step results, completions)

State is restored on restart — the mesh picks up where it left off.

---

15. Troubleshooting

Issue	Solution
`mesh: command not found`	Run from source: `node packages/cli/dist/main.js`
AI commands fail	Set `OPENMESH_LLM_BASE_URL` and `OPENMESH_LLM_API_KEY`
No events firing	Check `mesh logs` — ensure observers are running
Dashboard blank	Verify `--dashboard` flag and port not in use
MCP tools not appearing	Run `mesh mcp serve` and check client config
Channel test fails	Verify tokens in `mesh.config.json` or env vars
Plugin won't load	Check `openmesh-plugin.json` exists with valid `entry`

OpenMesh — Declare what. AI figures out how.